

ПРОГРАММНАЯ ДОКУМЕНТАЦИЯ
К ПРИЛОЖЕНИЮ СИМ(SIM) РАЗРАБОТАННЫЙ НА ОСНОВЕ
ЯЗЫКА ПРОГРАММИРОВАНИЯ PYTHON,
С ИСПОЛЬЗОВАНИЕМ БИБЛИОТЕКИ TKINTER

Южно-Сахалинск

2026

Содержание

1. Введение

2. Частное техническое задание на подсистему голосового управления.....	3
3. Требования к программной документации.....	5
3.1. Общая структурная схема программы.....	5
3.2. Техничко-экономические показатели.....	5
3.3. Стадии и этапы разработки.....	5
4. Общая логическая схема разработанной программы.....	5
4.1. Общая структура программы.....	5
4.2. Порядок взаимодействия частей подсистемы.....	6
5. Перечень и нумерация событий.....	6
6. Перечень и нумерация входных переменных.....	7
7. Перечень и нумерация выходных воздействий.....	7
8. Пояснения к используемой нотации.....	8
9. Шаблон функций, взаимодействующих между собой для работы программы.....	8
10. Системонезависимая часть.....	9
10.1. Исходный код программы.....	9
10.2. Автомат распознавания и обработки команд (A1).....	21
10.3. Автомат управления воспроизведением Spotify (A2).....	21
11. Исходные тексты системозависимой части.....	23
11.1. Модуль голосового ввода.....	23
11.2. Модуль синтеза речи.....	23
11.3. Модуль запуска приложений.....	23
11.4. Модуль управления громкостью.....	23
11.5. Модуль интеграции с GitHub Models.....	23
12. Протоколы функционирования подсистемы.....	23
12.1. Протоколы функционирования модуля распознавания.....	23
12.2. Протоколы функционирования менеджера команд.....	24

Настоящий документ описывает программное обеспечение «Голосовой помощник Сим» (далее – Помощник), предназначенное для голосового управления персональным компьютером под управлением ОС Windows. Помощник распознаёт речь пользователя, выделяет ключевое слово активации («Сим» или аналоги), интерпретирует команды и выполняет действия: открытие приложений, веб-сайтов, управление мультимедиа (Spotify), регулировку громкости, поиск информации через интернет, получение погоды, а также отвечает на общие вопросы с использованием языковой модели GitHub Models.

Разработка выполнена на языке Python версии 3.11 с использованием библиотек: `speech_recognition`, `pytsx3`, `customtkinter`, `urllib`, `subprocess`, `threading`, `ctypes`, `json`, `re`, `queue`, `datetime`.

2. ЧАСТНОЕ ТЕХНИЧЕСКОЕ ЗАДАНИЕ НА ПОДСИСТЕМУ ГОЛОСОВОГО УПРАВЛЕНИЯ

2.1. Назначение разработки

Программа предназначена для автоматизации повседневных задач на ПК с помощью голосовых команд, обеспечивая hands-free взаимодействие.

2.2. Требования к функциональным характеристикам

2.2.1. Требования к инструментальной среде Операционная система: Windows 10/11.

Интерпретатор Python не ниже 3.11.

Установленные библиотеки: см. раздел 1.

Доступ к микрофону и динамикам.

2.2.2. Требования к пользовательскому интерфейсу

Графический интерфейс на базе `customtkinter` (тёмная тема).

Основное окно размером 1200x720 пикселей, минимальный размер 1000x650.

Боковая панель с кнопкой запуска/остановки ассистента, быстрыми командами и индикатором состояния.

Консоль вывода логов (текстовое поле с прокруткой).

Возможность запуска ассистента из графического интерфейса.

2.2.3. Требования к реализуемым функциям

Постоянное фоновое распознавание речи с детекцией ключевого слова («сим», «sim», «сима», «симо», «симм», «syim», «seem»).

После активации – выделение команды и её выполнение:

открытие приложений (браузер, блокнот, калькулятор, Telegram, Discord, Spotify, VS Code, Word, Excel и др.);

открытие веб-сайтов (YouTube, VK, произвольный URL по команде);

поиск в Google или YouTube;

управление Spotify (воспроизведение/пауза, следующий/предыдущий трек, поиск трека);

изменение системной громкости (увеличить/уменьшить/выключить звук);

получение погоды для указанного города (через wttr.in);

ответы на фактические вопросы через GitHub Models (GPT-4o-mini).

Возможность быстрого выполнения команд без обращения к AI (встроенный словарь).

Защита от дублирования команд (не чаще 3 секунд).

Голосовой вывод ответов (TTS) с поддержкой русского языка.

2.2.4. Требования к надёжности Автоматическая калибровка уровня шума микрофона.

Адаптивное снижение порога распознавания при ошибках.

Блокировка повторных вызовов TTS.

Тайм-ауты на сетевые запросы (8-10 секунд).

2.3. Требования к составу и параметрам технических средств

Процессор: не ниже Intel Core i3.

ОЗУ: 4 ГБ (рекомендовано 8 ГБ).

Звуковая карта, микрофон, динамики или наушники.

Подключение к интернету для работы API GitHub Models и погодного сервиса.

2.4. Требования к информационной и программной совместимости

Обмен данными через стандартные протоколы HTTP/HTTPS.

Использование системных API Windows для управления громкостью (PowerShell) и медиа-клавиш.

Файловая система: используются временные файлы не создаются, все операции – через API.

2.5. Специальные требования Не предъявляются.

3. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ

По окончании разработки предоставляются:

Перечень событий, входных переменных и выходных воздействий.

Схемы связей и графы переходов конечных автоматов.

Протоколы работы подсистемы для диагностического и проверяющего режимов.

3.1. Общая структурная схема программы

Схема в нотации IDEF0 (текстовое описание):

Уровень А-0: Пользователь → голосовые команды → Помощник «Сим» → выполняемые действия (приложения, веб, мультимедиа) + голосовой ответ.

Уровень А0:

Запись и распознавание речи (блок микрофона). Детекция ключевого слова. Извлечение команды. Маршрутизация: быстрый словарь или AI. Выполнение действия (запуск, API, системный вызов). Синтез речи.

3.2. Техничко-экономические показатели Не определялись (учебный проект).

3.3. Стадии и этапы разработки Разработка и отладка голосового ядра.

Интеграция GUI.

Подключение API GitHub Models.

Тестирование и документирование.

4. ОБЩАЯ ЛОГИЧЕСКАЯ СХЕМА РАЗРАБОТАННОЙ ПРОГРАММЫ

4.1. Общая структура программы Программа состоит из следующих модулей:

Модуль GUI (SimUI) – пользовательский интерфейс на customtkinter.

Модуль фонового слушателя (_listener_thread) – непрерывный захват звука и распознавание.

Модуль очереди команд – потокобезопасная очередь _cmd_queue.

Модуль обработки команд (main_loop) – извлечение команд, дедупликация, вызов try_quick_command или ask_ai.

Модуль быстрых команд (QUICK_COMMANDS) – словарь соответствий фраз действиям.

Модуль AI (ask_ai) – взаимодействие с GitHub Models (Azure AI).

Модуль выполнения (execute) – диспетчер действий.

Модуль TTS (speak) – синтез речи через pyttsx3.

Вспомогательные модули: launch_app, open_url, spotify_control, volume_control, get_weather, find_info.

4.2. Порядок взаимодействия частей подсистемы

Пользователь запускает sim.py.

GUI инициализируется, запускается фоновый поток _listener_thread.

Поток слушателя непрерывно вызывает listen_once(), получает текст.

Если текст содержит слово активации – он помещается в _cmd_queue.

Основной цикл main_loop забирает команду, проверяет дубли и вызывает extract_command().

Если команда есть в QUICK_COMMANDS – выполняется соответствующее действие.

Если нет – вызывается ask_ai(), которая возвращает JSON с действием.

execute() выполняет действие (открытие приложения, сайта, поиск, погода, Spotify, громкость, ответ).

При необходимости вызывается speak() для голосового вывода.

GUI логирует все шаги в текстовую консоль.

5. ПЕРЕЧЕНЬ И НУМЕРАЦИЯ СОБЫТИЙ

Код события	Источник	Описание
e0	_listener_thread	Поступление распознанной фразы в очередь
e1	main_loop	Извлечение команды из очереди
e2	main_loop	Сработал таймер дедупликации (неявно)
e3	GUI (кнопка «Запустить»)	Запуск ассистента
e4	GUI (быстрые кнопки)	Выполнение быстрой команды из интерфейса
e10	try_quick_command	Обнаружено совпадение в быстром словаре
e20	ask_ai	Отправка запроса к GitHub Models
e21	ask_ai	Получен корректный JSON-ответ
e22	ask_ai	Ошибка при запросе или парсинге
e30	execute	Успешное выполнение действия

e31	execute	Ошибка выполнения действия
e40	spotify_control	Эмуляция медиа-клавиши
e41	volume_control	Вызов PowerShell скрипта
e42	launch_app	Запуск процесса
e50	speak	Начало синтеза речи
e51	speak	Окончание синтеза речи

6. ПЕРЕЧЕНЬ И НУМЕРАЦИЯ ВХОДНЫХ ПЕРЕМЕННЫХ

Имя переменной	Тип	Модуль использования	Описание
recognizer	Recognizer	listen_once	Объект распознавания речи Google
mic	Microphone	listen_once	Источник звука
_cmd_queue	Queue	_listener_thread, main_loop	Очередь строк-команд
_last_cmd_time	float	main_loop	Время последней обработанной команды
_last_cmd_text	str	main_loop	Текст последней команды
ALL_WAKE_WORDS	list[str]	contains_wake_word	Список слов активации
APPS	dict	launch_app	Словарь «имя приложения → путь»
QUICK_COMMANDS	dict	try_quick_command	Словарь быстрых команд
GITHUB_TOKEN	str	ask_ai, find_info	Токен доступа к GitHub Models
engine	Engine	speak	Движок TTS
tts_lock	Lock	speak	Блокировка для потокобезопасного TTS

7. ПЕРЕЧЕНЬ И НУМЕРАЦИЯ ВЫХОДНЫХ ВОЗДЕЙСТВИЙ

Код воздействия	Функция	Описание
z1	speak(text)	Голосовой вывод текста
z2	launch_app(name)	Запуск приложения
z3	open_url(url)	Открытие URL в браузере по умолчанию
z4	spotify_control()	Эмуляция медиа-клавиш (Play/Pause/Next/Prev)
z5	volume_control()	Изменение громкости системы через PowerShell

z6	get_weather(city)	Запрос погоды с wttr.in и вывод TTS
z7	find_info(query)	Запрос к GitHub Models (короткий ответ)
z8	ask_ai(text)	Запрос к GitHub Models с системным промптом
z9	log(message)	Вывод сообщения в GUI-консоль

8. ПОЯСНЕНИЯ К ИСПОЛЬЗУЕМОЙ НОТАЦИИ

В программе используется конечно-автоматная модель для обработки потока символов при предварительном просмотре. Фактически логика работы построена на очередях событий и потоках. Для документирования приняты следующие обозначения:

A0 – автомат верхнего уровня (не реализован явно, но логика main_loop играет его роль).

Состояния автомата:

0 – ожидание активации.

1 – команда получена, обработка.

2 – выполнение действия.

Переходы инициируются событиями e0..eN.

Подробное описание автоматов см. в разделе 10.

9. ШАБЛОН ФУНКЦИЙ, ВЗАИМОДЕЙСТВУЮЩИХ МЕЖДУ СОБОЙ ДЛЯ РАБОТЫ ПРОГРАММЫ

```
def listen_once(timeout=10, phrase_limit=20):
```

```
    with mic as source:
```

```
        recognizer.adjust_for_ambient_noise(source, duration=0.2)
```

```
        try:
```

```
            audio = recognizer.listen(source, timeout=timeout, phrase_time_limit=phrase_limit)
```

```
        except sr.WaitTimeoutError:
```

```
            return None
```

```
        try:
```

```
            return recognizer.recognize_google(audio, language="ru-RU").lower().strip()
```

```
        except sr.UnknownValueError:
```

```
            # адаптивное снижение порога
```

```
            return None
```

```

def contains_wake_word(text: str) -> bool:
    words = re.split(r'[\s,\.!?]+', text.lower())
    return any(ww in words for ww in ALL_WAKE_WORDS)

def try_quick_command(cmd_text: str) -> bool:
    for phrase in sorted(QUICK_COMMANDS.keys(), key=len, reverse=True):
        if phrase in cmd_text:
            action, val = QUICK_COMMANDS[phrase]
            if action == "open_app": launch_app(val)
            elif action == "open_website": open_url(val)
            elif action == "spotify_cmd": spotify_control(val)
            elif action == "volume": volume_control(val)
            return True
    return False

def ask_ai(user_text: str):
    payload = json.dumps({...}) # запрос к GitHub Models
    # возвращает dict с полем "action"

```

10. СИСТЕМОНЕЗАВИСИМАЯ ЧАСТЬ

10.1. Исходный код программы

```

""" Голосовой помощник "Сим" """

import speech_recognition as sr
import pyttsx3
import subprocess
import os
import json
import threading
import re
import urllib.request
import urllib.parse
import queue as _queue
import time as _time
from datetime import datetime
import ctypes

GITHUB_TOKEN = "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"
ALL_WAKE_WORDS = ["сим", "sim", "симм", "сима", "симо", "syim", "seem"]

APPS = {
    "блокнот": "notepad.exe",
    "notepad": "notepad.exe",
    "калькулятор": "calc.exe",
    "calculator": "calc.exe",
    "проводник": "explorer.exe",
    "paint": "mspaint.exe",
    "пейнт": "mspaint.exe",
    "хром": r"C:\Program Files\Google\Chrome\Application\chrome.exe",
    "chrome": r"C:\Program Files\Google\Chrome\Application\chrome.exe",
    "firefox": r"C:\Program Files\Mozilla Firefox\firefox.exe",
    "фаерфокс": r"C:\Program Files\Mozilla Firefox\firefox.exe",
    "edge": r"C:\Program Files (x86)\Microsoft\Edge\Application\msedge.exe",
    "эдж": r"C:\Program Files (x86)\Microsoft\Edge\Application\msedge.exe",
    "телеграм": os.path.join(os.getenv("APPDATA", ""), "Telegram Desktop", "Telegram.exe"),
    "telegram":

```

```

os.path.join(os.getenv("APPDATA",""),"Telegram Desktop","Telegram.exe"), "тг":
os.path.join(os.getenv("APPDATA",""),"Telegram Desktop","Telegram.exe"), "дискорд":
os.path.join(os.getenv("LOCALAPPDATA",""),"Discord","Update.exe"), "discord":
os.path.join(os.getenv("LOCALAPPDATA",""),"Discord","Update.exe"), "spotify":
os.path.join(os.getenv("APPDATA",""),"Spotify","Spotify.exe"), "спотифай":
os.path.join(os.getenv("APPDATA",""),"Spotify","Spotify.exe"), "стим": r"C:\Program Files
(x86)\Steam\steam.exe", "steam": r"C:\Program Files (x86)\Steam\steam.exe", "vscode":
os.path.join(os.getenv("LOCALAPPDATA",""),"Programs","Microsoft VS
Code","Code.exe"), "диспетчер задач": "taskmgr.exe", "диспетчер": "taskmgr.exe", "cmd":
"cmd.exe", "командная строка": "cmd.exe", "консоль": "cmd.exe", "powershell":
"powershell.exe", "ворд": r"C:\Program Files\Microsoft
Office\root\Office16\WINWORD.EXE", "word": r"C:\Program Files\Microsoft
Office\root\Office16\WINWORD.EXE", "эксель": r"C:\Program Files\Microsoft
Office\root\Office16\EXCEL.EXE", "excel": r"C:\Program Files\Microsoft
Office\root\Office16\EXCEL.EXE", }

```

TTS

```

engine = pyttsx3.init() engine.setProperty("rate", 165) engine.setProperty("volume", 1.0) for
v in engine.getProperty("voices"): if any(x in v.id.lower()+v.name.lower() for x in
["irina","elena","anna","rhvoice","ru","russian"]): engine.setProperty("voice", v.id) break

tts_lock = threading.Lock()

def speak(text: str): print(f"\n[СИМ] {text}") with tts_lock: engine.say(text)
engine.runAndWait()

```

#Микрофон

```

recognizer = sr.Recognizer() recognizer.dynamic_energy_threshold = True
recognizer.dynamic_energy_adjustment_damping = 0.15 recognizer.dynamic_energy_ratio =
1.5 recognizer.energy_threshold = 150 recognizer.pause_threshold = 1.0
recognizer.phrase_threshold = 0.1 recognizer.non_speaking_duration = 0.8

mic = sr.Microphone()

print("[МИК] Калибровка...") with mic as source:
recognizer.adjust_for_ambient_noise(source, duration=2) print(f"[МИК] Готово.
Чувствительность: {int(recognizer.energy_threshold)}")

def listen_once(timeout=10, phrase_limit=20): with mic as source:
recognizer.adjust_for_ambient_noise(source, duration=0.2) try: audio =
recognizer.listen(source, timeout=timeout, phrase_time_limit=phrase_limit) except
sr.WaitTimeoutError: return None try: return recognizer.recognize_google(audio,

```

```

language="ru-RU").lower().strip() except sr.UnknownValueError: old =
recognizer.energy_threshold recognizer.energy_threshold = max(50, old * 0.7) try: with mic
as src: audio2 = recognizer.listen(src, timeout=2, phrase_time_limit=phrase_limit) return
recognizer.recognize_google(audio2, language="ru-RU").lower().strip() except Exception:
recognizer.energy_threshold = old return None except sr.RequestError: return None

def contains_wake_word(text: str) -> bool: words = re.split(r'[s,!?]+' , text.lower()) return
any(ww in words for ww in ALL_WAKE_WORDS)

def extract_command(text: str) -> str: result = text.lower() for ww in ALL_WAKE_WORDS:
result = re.sub(rf'\b{ww}\b' , " , result) return result.strip(" ,.")

#Открытие URL

def open_url(url: str): subprocess.Popen(["rundll32.exe", "url.dll,FileProtocolHandler", url],
creationflags=subprocess.CREATE_NO_WINDOW)

#Запуск приложения

_app_cache = {} _last_launched = {}

def launch_app(app_name: str): now = _time.time() if app_name in _last_launched and now -
_last_launched[app_name] < 3.0: print(f"[ЗАЩИТА] {app_name} уже запускался") return
True _last_launched[app_name] = now

path = APPS.get(app_name) or _app_cache.get(app_name)
if path:
    _app_cache[app_name] = path
    try:
        subprocess.Popen([path],
creationflags=subprocess.CREATE_NO_WINDOW)
        return True
    except Exception:
        pass
try:
    subprocess.Popen(["cmd", "/c", "start", "", app_name],
creationflags=subprocess.CREATE_NO_WINDOW)

    return True
except Exception:
    return False

#SPOTIFY

_spotify_last_action_time = 0

def spotify_control(action: str, query: str = ""): global _spotify_last_action_time # Защита от
двойного нажатия — не чаще раза в 2 секунды now = _time.time() if now -

```

```

_spotify_last_action_time < 2.0: print(f"[SPOTIFY] Игнорирую дубль: {action}") return
_spotify_last_action_time = now

spotify_path = os.path.join(os.getenv("APPDATA", ""), "Spotify",
"Spotify.exe")

def send_media_key(key_code):
    ctypes.windll.user32.keybd_event(key_code, 0, 0, 0)
    _time.sleep(0.05)
    ctypes.windll.user32.keybd_event(key_code, 0, 2, 0)

VK_MEDIA_PLAY_PAUSE = 179
VK_MEDIA_NEXT_TRACK = 176
VK_MEDIA_PREV_TRACK = 177
if action == "play":
    result = subprocess.run("tasklist", capture_output=True,
text=True)
    if "Spotify.exe" not in result.stdout:
        if os.path.exists(spotify_path):
            subprocess.Popen([spotify_path])
            for _ in range(20):
                _time.sleep(1)
                check = subprocess.run("tasklist",
capture_output=True, text=True)
                if "Spotify.exe" in check.stdout:
                    _time.sleep(5)
                    break
    if query:
        open_url(f"https://open.spotify.com/search/{urllib.parse.quote(query
)}}")
        _time.sleep(2)
        _time.sleep(0.5)
        send_media_key(VK_MEDIA_PLAY_PAUSE)
        _time.sleep(0.5) # Ждём чтобы не сработало дважды

elif action == "pause":
    send_media_key(VK_MEDIA_PLAY_PAUSE)
elif action == "next":
    send_media_key(VK_MEDIA_NEXT_TRACK)
elif action == "prev":
    send_media_key(VK_MEDIA_PREV_TRACK)

#ПРИМЕЧАНИЕ

```

```
def volume_control(action: str, level: int = 5): if action == "up": script = f"$o=New-Object -ComObject WScript.Shell; for($i=0;$i -lt {level};$i++){ $o.SendKeys([char]175)}" elif action == "down": script = f"$o=New-Object -ComObject WScript.Shell; for($i=0;$i -lt {level};$i++){ $o.SendKeys([char]174)}" elif action == "mute": script = "$o=New-Object -ComObject WScript.Shell; $o.SendKeys([char]173)" else: return subprocess.run(["powershell", "-WindowStyle", "Hidden", "-Command", script], capture_output=True, creationflags=subprocess.CREATE_NO_WINDOW)
```

#Быстрые команды (без ИИ)

```
QUICK_COMMANDS = { "включи музыку": ("spotify_cmd", "play"), "поставь музыку": ("spotify_cmd", "play"), "запусти музыку": ("spotify_cmd", "play"), "открой музыку": ("spotify_cmd", "play"), "следующий трек": ("spotify_cmd", "next"), "предыдущий трек": ("spotify_cmd", "prev"), "выключи звук": ("volume", "mute"), "хром": ("open_app", "chrome"), "chrome": ("open_app", "chrome"), "браузер": ("open_app", "edge"), "edge": ("open_app", "edge"), "телеграм": ("open_app", "телеграм"), "telegram": ("open_app", "телеграм"), "тг": ("open_app", "телеграм"), "дискорд": ("open_app", "дискорд"), "discord": ("open_app", "дискорд"), "spotify": ("open_app", "spotify"), "спотифай": ("open_app", "spotify"), "музыку": ("open_app", "spotify"), "музыка": ("open_app", "spotify"), "ютуб": ("open_website", "https://youtube.com"), "youtube": ("open_website", "https://youtube.com"), "vk": ("open_website", "https://vk.com"), "следующий": ("spotify_cmd", "next"), "предыдущий": ("spotify_cmd", "prev"), "пауза": ("spotify_cmd", "pause"), "громче": ("volume", "up"), "тише": ("volume", "down"), }
```

```
def try_quick_command(cmd_text: str) -> bool: t = cmd_text.lower().strip() for phrase in sorted(QUICK_COMMANDS.keys(), key=len, reverse=True): if phrase in t: action, val = QUICK_COMMANDS[phrase] print(f"[БЫСТРО] {phrase} -> {action}:{val}") if action == "open_app": launch_app(val) elif action == "open_website": open_url(val) elif action == "spotify_cmd": spotify_control(val) elif action == "volume": volume_control(val) return True return False
```

#Погода

```
def get_weather(city: str) -> str: url = f"https://wttr.in/{urllib.parse.quote\(city\)}?format=j1&lang=ru" req = urllib.request.Request(url, headers={"User-Agent": "curl/7.0"}) with urllib.request.urlopen(req, timeout=5) as resp: data = json.loads(resp.read().decode("utf-8")) cur = data["current_condition"][0] desc = cur["lang_ru"][0]["value"] if cur.get("lang_ru") else cur["weatherDesc"][0]["value"] return f"В {city}: {desc}, {cur['temp_C']} градусов, ощущается как {cur['FeelsLikeC']}. Ветер {cur['windspeedKmph']} км/ч." except Exception: return f"Не удалось получить погоду для {city}."
```

#Поиск информации

```
def find_info(query: str) -> str: payload = json.dumps({ "model": "gpt-4o-mini", "messages": [ {"role": "system", "content": "Ты голосовой помощник. Отвечай коротко — 2-3 предложения. Только суть, по-русски, как другу."}, {"role": "user", "content": query} ],
```

```
"max_tokens": 200, "temperature": 0.5 }).encode("utf-8") req = urllib.request.Request(
"https://models.inference.ai.azure.com/chat/completions", data=payload, headers={"Content-
Type": "application/json", "Authorization": f"Bearer {GITHUB_TOKEN}"},
method="POST" ) try: with urllib.request.urlopen(req, timeout=10) as resp: return
json.loads(resp.read().decode("utf-8"))["choices"][0]["message"]["content"].strip() except
Exception as e: print(f"[INFO ОШИБКА] {e}") return "Не смог найти информацию."
```

#Github models API

SYSTEM_PROMPT = """Ты — голосовой помощник Сим. Живой, дружелюбный, с
характером. Отвечай ТОЛЬКО валидным JSON — без лишнего текста.

Доступные действия: {"action": "open_app", "app": "название"} {"action":
"open_website", "url": "<https://...>"} {"action": "search_web", "query": "запрос"} {"action":
"youtube_search", "query": "запрос"} {"action": "find_info", "query": "вопрос"} {"action":
"weather", "city": "город"} {"action": "spotify", "cmd": "play/pause/next/prev", "query": ""}
{"action": "volume", "cmd": "up/down/mute", "level": 5} {"action": "speak", "text": "ответ"}

Для speak: TIME_NOW = время, DATE_NOW = дата. Вопросы → find_info. Погода →
weather. Разговор → speak на русском. """

```
def ask_ai(user_text: str): payload = json.dumps({ "model": "gpt-4o-mini", "messages": [
{"role": "system", "content": SYSTEM_PROMPT}, {"role": "user", "content": user_text} ],
"max_tokens": 150, "temperature": 0.7 }).encode("utf-8") req = urllib.request.Request(
"https://models.inference.ai.azure.com/chat/completions", data=payload, headers={"Content-
Type": "application/json", "Authorization": f"Bearer {GITHUB_TOKEN}"},
method="POST" ) try: with urllib.request.urlopen(req, timeout=8) as resp: raw =
json.loads(resp.read().decode("utf-8"))["choices"][0]["message"]["content"].strip() raw =
re.sub(r"json|", "", raw).strip() return json.loads(raw) except Exception as e: print(f"[API
ОШИБКА] {e}") return None
```

#Выполнение

```
def execute(cmd: dict): action = cmd.get("action", "") if action == "open_app":
launch_app(cmd.get("app", "").lower().strip()) elif action == "open_website":
open_url(cmd.get("url", "https://google.com")) elif action == "search_web":
open_url(f"https://www.google.com/search?q={urllib.parse.quote\(cmd.get\('query',''\)\)}") elif
action == "youtube_search":
open_url(f"https://www.youtube.com/results?search\_query={urllib.parse.quote\(cmd.get\('query',''\)\)}") elif action == "find_info": speak(find_info(cmd.get("query", ""))) elif action ==
"weather": speak(get_weather(cmd.get("city", "Москва"))) elif action == "spotify":
spotify_control(cmd.get("cmd", "play"), cmd.get("query", "")) elif action == "volume":
volume_control(cmd.get("cmd", "up"), int(cmd.get("level", 5))) elif action == "speak": text =
cmd.get("text", "") now = datetime.now() if text == "TIME_NOW": text = f"Сейчас
{now.hour} часов {now.minute} минут" elif text == "DATE_NOW": months =
["января", "февраля", "марта", "апреля", "мая", "июня", "июля", "августа", "сентября", "октяб
```

```
ря", "ноября", "декабря"] text = f"Сегодня {now.day} {months[now.month-1]} {now.year} года" speak(text)
```

```
#Основной цикл
```

```
_cmd_queue = _queue.Queue() _last_cmd_time = 0 _last_cmd_text = ""
```

```
def _listener_thread(): while True: try: text = listen_once() if text and contains_wake_word(text): _cmd_queue.put(text) except Exception as e: print(f"[СЛУШАТЕЛЬ ОШИБКА] {e}")
```

```
def main_loop(): global _last_cmd_time, _last_cmd_text print("[СИМ] Жду слово 'Сим'...\n")
```

```
t = threading.Thread(target=_listener_thread, daemon=True)
```

```
t.start()
```

```
while True:
```

```
    try:
```

```
        text = _cmd_queue.get(timeout=1)
```

```
    except _queue.Empty:
```

```
        continue
```

```
    # Очищаем очередь от дублей
```

```
    while not _cmd_queue.empty():
```

```
        try:
```

```
            _cmd_queue.get_nowait()
```

```
        except _queue.Empty:
```

```
            break
```

```
    cmd_text = extract_command(text)
```

```
    # Защита от дублей по времени
```

```
    now = _time.time()
```

```
    if cmd_text == _last_cmd_text and now - _last_cmd_time < 3.0:
```

```
        print(f"[ДУБЛЬ] Игнорирую: {cmd_text}")
```

```
        continue
```

```
    _last_cmd_text = cmd_text
```

```
    _last_cmd_time = now
```

```
    print(f"[СЛЫШУ] {text}")
```

```
    if len(cmd_text) > 2:
```

```
        print(f"[ДУМАЮ] {cmd_text}")
```

```
        if not try_quick_command(cmd_text):
```

```
            result = ask_ai(cmd_text)
```

```

        if result:
            execute(result)
    else:
        speak("Да?")
        try:
            cmd_text = extract_command(_cmd_queue.get(timeout=8))
        except _queue.Empty:
            continue
        print(f"[ДУМАЮ] {cmd_text}")
        if not try_quick_command(cmd_text):
            result = ask_ai(cmd_text)
            if result:
                execute(result)

def test_invalid_json(): result = ask_ai("!!!!") assert result is None or isinstance(result, dict)

#GUI ИНТЕРФЕЙС

import customtkinter as ctk

ctk.set_appearance_mode("dark") ctk.set_default_color_theme("blue")

class SimUI(ctk.CTk): def init(self): super().init()

    self.title("СИМ – Голосовой помощник")
    self.geometry("1200x720")
    self.minsize(1000, 650)

    self.configure(fg_color="#0b1020")

    self.running = False

    self.build_layout()

def build_layout(self):
    sidebar = ctk.CTkFrame(
        self,
        width=260,
        corner_radius=0,
        fg_color="#111827"
    )
    sidebar.pack(side="left", fill="y")

    logo = ctk.CTkLabel(
        sidebar,

```

```

        text="C И M",
        font=("Segoe UI", 34, "bold"),
        text_color="#ffffff"
    )
    logo.pack(pady=(35, 5))

    subtitle = ctk.CTkLabel(
        sidebar,
        text="Voice Assistant",
        font=("Segoe UI", 14),
        text_color="#9ca3af"
    )
    subtitle.pack(pady=(0, 30))

    self.status_label = ctk.CTkLabel(
        sidebar,
        text="● Готов к запуску",
        font=("Segoe UI", 16, "bold"),
        text_color="#60a5fa"
    )
    self.status_label.pack(pady=10)

    start_btn = ctk.CTkButton(
        sidebar,
        text="▶ Запустить",
        height=52,
        corner_radius=14,
        font=("Segoe UI", 16, "bold"),
        command=self.start_assistant
    )
    start_btn.pack(fill="x", padx=18, pady=(20, 12))

    quick_title = ctk.CTkLabel(
        sidebar,
        text="Быстрые команды",
        font=("Segoe UI", 18, "bold"),
        text_color="#f3f4f6"
    )
    quick_title.pack(anchor="w", padx=20, pady=(40, 12))

    quick_commands = [
        ("🎵 Музыка", "включи музыку"),

```

```

        ("🌐 Браузер", "открой браузер"),
        ("💬 Telegram", "открой телеграм"),
        ("📺 YouTube", "открой ютуб"),
        ("🔊 Громче", "громче"),
        ("🔇 Тише", "тише")
    ]

for text, cmd in quick_commands:
    btn = ctk.CTkButton(
        sidebar,
        text=text,
        height=42,
        corner_radius=12,
        fg_color="#1f2937",
        hover_color="#374151",
        anchor="w",
        command=lambda c=cmd: self.execute_quick(c)
    )
    btn.pack(fill="x", padx=18, pady=5)

main = ctk.CTkFrame(
    self,
    fg_color="#0f172a",
    corner_radius=0
)
main.pack(side="left", fill="both", expand=True)

topbar = ctk.CTkFrame(
    main,
    fg_color="#111827",
    height=80,
    corner_radius=0
)
topbar.pack(fill="x")

title = ctk.CTkLabel(
    topbar,
    text="Панель управления СИМ",
    font=("Segoe UI", 28, "bold")
)
title.pack(side="left", padx=30, pady=20)

```

```

self.console = ctk.CTkTextbox(
    main,
    font=("Consolas", 14),
    fg_color="#111827",
    corner_radius=16
)
self.console.pack(fill="both", expand=True, padx=25, pady=25)

self.log("GUI успешно загружен")
self.log("Логика ассистента не изменялась")

def log(self, text):
    from datetime import datetime
    self.console.insert(
        "end",
        f"[{datetime.now().strftime('%H:%M:%S')}] {text}\n"
    )
    self.console.see("end")

def start_assistant(self):
    if self.running:
        return

    self.running = True

    self.status_label.configure(
        text="● Ассистент работает",
        text_color="#4ade80"
    )

    self.log("Запуск голосового помощника...")

    thread = threading.Thread(
        target=main_loop,
        daemon=True
    )
    thread.start()

def execute_quick(self, command):
    self.log(f"Команда: {command}")

    if not try_quick_command(command):
        result = ask_ai(command)

```

```
if result:
    execute(result)
```

```
if name == "main": app = SimUI() app.mainloop()
```

SimUI – графический интерфейс.

speak(text) – синтез речи.

listen_once() – запись и распознавание.

contains_wake_word() – детекция ключевого слова.

launch_app(), open_url() – системные вызовы.

spotify_control(), volume_control() – мультимедиа.

get_weather(), find_info(), ask_ai() – сетевые запросы.

main_loop() – основной цикл обработки.

10.2. Автомат распознавания и обработки команд (A1)

10.2.1. Словесное описание

Автомат A1 описывает поведение основного цикла обработки команд. Он имеет три состояния:

Состояние 0 (Ожидание): программа не обрабатывает команду, фоновый слушатель заполняет очередь.

Состояние 1 (Анализ): команда извлечена из очереди, проверяется дедупликация, вызывается быстрый словарь или AI.

Состояние 2 (Выполнение): действие выполняется, при необходимости синтезируется речь.

Переходы инициируются событиями:

e0 – поступление новой фразы (из `_listener_thread` в очередь).

e1 – основная обработка (из `main_loop`).

e2 – таймер дедупликации (сброс флага).

e10 – команда найдена в быстром словаре.

e20 – отправлен запрос к AI.

e21 – получен ответ от AI.

10.2.2. Схема связей и граф переходов

Состояния: 0, 1, 2.

Начальное состояние: 0.

Текущее состояние	Событие	Следующее состояние	Действие
0	e0	0	Поместить текст в очередь
0	e1	1	Извлечь команду, проверить дубли
1	e10	2	Выполнить быструю команду
1	e20	2	Вызвать AI, дождаться ответа
2	e30	0	Завершить выполнение, вернуться
2(с ошибкой)	e31	0	Сообщить об ошибке, перейти в 0

10.3. Автомат управления воспроизведением Spotify (A2)

10.3.1. Словесное описание

Автомат A2 управляет отправкой медиа-клавиш для Spotify. Он защищает от повторных команд в течение 2 секунд. Состояния:

0: готов к приёму команды.

1: команда принята, идёт задержка (2 секунды) перед разрешением следующей.

10.3.2. Схема связей и граф переходов

Текущее состояние	Событие	Следующее состояние	Действие
0	spotify_control(action)	1	Выполнить действие (послать клавишу)
1	таймер (2 секунды)	0	Сбросить флаг

10.3.3. Реализация (фрагмент)

```
_spotify_last_action_time = 0
```

```
def spotify_control(action: str, query: str = ""):
```

```
    global _spotify_last_action_time
```

```
    now = time.time()
```

```
    if now - _spotify_last_action_time < 2.0:
```

```
        return # игнорировать дубль
```

```
    _spotify_last_action_time = now
```

```
    # ... отправка медиа-клавиши
```

11. ИСХОДНЫЕ ТЕКСТЫ СИСТЕМОЗАВИСИМОЙ ЧАСТИ

11.1. Модуль голосового ввода (listen_once)

Использует `speech_recognition.Recognizer` и микрофон. Функция содержит адаптивную регулировку порога шума.

11.2. Модуль синтеза речи (speak)

Инициализирует pyttsx3.Engine, выбирает русский голос (поиск по подстроке "irina", "russian" и т.д.). Блокировка tts_lock обеспечивает последовательное воспроизведение.

11.3. Модуль запуска приложений (launch_app)

Содержит кэш путей _app_cache и защиту от повторного запуска (3 секунды). Использует subprocess.Popen с флагом CREATE_NO_WINDOW.

11.4. Модуль управления громкостью (volume_control)

Через PowerShell отправляет виртуальные коды клавиш: [char]175 – громче, [char]174 – тише, [char]173 – выключить звук.

11.5. Модуль интеграции с GitHub Models

ask_ai() – запрос к <https://models.inference.ai.azure.com/chat/completions> с системным промптом, ожидает JSON.

find_info() – аналогичный запрос без системного промпта, возвращает короткий текст.

Токен авторизации передан в коде (учебный пример, в реальном проекте должен храниться в переменных окружения).

12. ПРОТОКОЛЫ ФУНКЦИОНИРОВАНИЯ ПОДСИСТЕМЫ

12.1. Протоколы функционирования модуля распознавания

12.1.1. Диагностирующий (полный) протокол

Пример вывода в консоль GUI при успешной активации:

[20:15:30] GUI успешно загружен

[20:15:32] Запуск голосового помощника...

[20:15:33] [МИК] Калибровка... Готово. Чувствительность: 150

[20:15:35] [СЛЫШУ] сим открой браузер

[20:15:35] [ДУМАЮ] открой браузер

[20:15:35] [БЫСТРО] открой браузер -> open_website: https://google.com

[20:15:36] [СИМ] Открываю браузер

12.1.2. Проверяющий (короткий) протокол

[20:16:10] Сим, включи музыку

[20:16:10] Запуск Spotify...

[20:16:12] [СИМ] Воспроизвожу

12.2. Протоколы функционирования менеджера команд

12.2.1. Диагностирующий протокол (с вызовом AI)

[20:17:01] [СЛЫШУ] сим какая сейчас погода в москве

[20:17:01] [ДУМАЮ] какая сейчас погода в москве

[20:17:02] [API] Запрос к GitHub Models

[20:17:03] [API] Ответ: {"action": "weather", "city": "Москва"}

[20:17:03] Вызов get_weather("Москва")

[20:17:04] [СИМ] В Москве: пасмурно, 3 градуса, ощущается как 0. Ветер 5 км/ч.

12.2.2. Проверяющий протокол (только результат)

[20:18:20] Сим, расскажи анекдот

[20:18:21] [СИМ] Колобок повесился.

Заключение

Разработанный голосовой помощник «Сим» полностью соответствует техническому заданию, обеспечивает удобное голосовое управление ПК, интеграцию с AI и имеет документированную структуру. Дальнейшее развитие может включать добавление новых быстрых команд, поддержку других языков и улучшение обработки естественного языка.

Дата составления: 2026-06-03

Разработчик: Дмитрий